

51-lks-competition-prep

□ LKS Web Technology — Competition Preparation



Figure 1: Banner

Level: Advanced

Prerequisites: Modul 4 (HTML/CSS), Modul 6 (JavaScript), Modul 11 (PHP/Backend), Modul 46 (Database)

Output: Siap mengikuti lomba Web Technology tingkat LKS SMK

Durasi: 3 sesi (+ 1 sesi mock test 6 jam)

□ Daftar Sesi

Sesi	Judul	Durasi	Materi
01	LKS Mapping & Gap Analysis	120 menit	Standar LKS, mapping kompetensi, identifikasi gap
02	Competition Strategy	120 menit	Time management, scoring criteria, mock test simulation
03	Mock Projects	240 menit	3 soal mock LKS lengkap dengan answer key

□ Tujuan Pembelajaran

Setelah menyelesaikan modul ini, peserta mampu:

1. **Memahami** standar kompetensi LKS Web Technology dan memetakannya terhadap kurikulum yang sudah dipelajari
2. **Mengidentifikasi** gap kompetensi pribadi dan menyusun rencana remedial
3. **Menerapkan** strategi time management dan prioritas saat lomba
4. **Menghindari** common pitfalls yang sering mengurangi skor
5. **Menyelesaikan** soal mock test LKS dalam batas waktu 4-6 jam

□ Software & Tools

Tool	Kegunaan
VS Code	Code editor utama
XAMPP / Laragon	Local web server + MySQL
MySQL Workbench / phpMyAdmin	Database management
draw.io / Figma	Perancangan layout & diagram
Postman / Thunder Client	API testing
Git + GitHub	Version control (opsional)

□ Aspek Penilaian LKS

Kriteria	Bobot	Deskripsi
Functional Correctness	35%	Semua fitur berfungsi sesuai spesifikasi
Code Quality	20%	Struktur kode, naming, documentation
Efficiency	15%	Performa query, load time, resource usage
UI/UX	15%	Responsiveness, konsistensi, user experience
Creativity	10%	Inovasi solusi di luar requirement dasar
Documentation	5%	README, code comments, user guide

□ Struktur Module

```
51-lks-competition-prep/  
├── README.md  
├── 01-lks-mapping.md      # Mapping standar LKS ke kurikulum  
├── 02-lks-strategy.md     # Strategi lomba & mock simulation  
└── 03-lks-mock-projects.md # 3 soal mock LKS + answer key
```

□ Referensi

- [LKS SMK — Web Technology](#)
- [WorldSkills — Web Technologies](#)
- Dokumentasi: MDN, PHP.net, Express.js, MySQL

□ Sesi 01 — LKS Mapping & Gap Analysis

Durasi: 120 menit

Tujuan: Memahami standar LKS Web Technology dan memetakan kompetensi yang sudah dikuasai vs yang perlu diperkuat

1. Standar Kompetensi LKS Web Technology

LKS Web Technology (berdasarkan WorldSkills Standard) terbagi atas 4 area utama:

□ Web Design (25%)

Sub-kompetensi	Detail
Responsive Design	Mobile-first, breakpoints, fluid layout
CSS Architecture	Flexbox, Grid, custom properties, animations
UI Consistency	Color system, typography, spacing, component design
Accessibility	ARIA labels, semantic HTML, keyboard navigation, contrast
Cross-browser	Vendor prefixes, fallbacks, progressive enhancement

□ Web Programming (35%)

Sub-kompetensi	Detail
Frontend Logic	DOM manipulation, event handling, async JS
Backend Logic	Server-side scripting, middleware, routing
API Integration	RESTful endpoints, JSON handling, fetch/AJAX
Form Handling	Validation (client + server), sanitization, file upload
Authentication	Session management, JWT, password hashing

□ Database (25%)

Sub-kompetensi	Detail
Schema Design	Normalization (1NF–3NF), ERD, relationships
SQL Queries	SELECT, JOIN, GROUP BY, subquery, indexing
CRUD Operations	Create, Read, Update, Delete via backend
Data Integrity	Constraints, transactions, prepared statements
Migration	Schema versioning, seeding, rollback

⚙ System Administration (15%)

Sub-kompetensi	Detail
Server Setup	Local dev environment, hosting configuration
Deployment	FTP/SSH, environment variables, domain config
Version Control	Git init, commit, branch, merge, conflict resolution
Security Basics	SQL injection prevention, XSS protection, HTTPS

2. Mapping Kurikulum ke Standar LKS

Modul yang Relevan

Modul	Topik	Coverage LKS
Modul 04	HTML & CSS Dasar	Web Design 40% — butuh CSS Grid/Flexbox + responsive
Modul 06	JavaScript Dasar	Web Programming 30% — butuh async, fetch, error handling
Modul 11	PHP & MySQL	Web Programming 40%, Database 50% — butuh prepared stmt

Modul	Topik	Coverage LKS
Modul 46	Database Design	Database 60% — butuh normalisasi + indexing

Gap Matrix

Kompetensi LKS	Status	Keterangan
Responsive Design	□ Gap	Belum dibahas secara mendalam
CSS Grid & Flexbox	□ Partial	Hanya dasar
Async JavaScript	□ Gap	Belum ada materi fetch/AJAX
REST API	□ Gap	Belum ada materi backend API
Prepared Statements	□ Partial	Disebut sekilas
Authentication	□ Gap	Belum ada session/JWT
Normalization	□ Partial	Konsep dasar ada
Deployment	□ Gap	Belum ada
Version Control	□ Partial	Git init saja
Accessibility	□ Gap	Belum disentuh

Prioritas Belajar

Prioritas	Kompetensi	Alasan
□ P0	REST API + AJAX	Paling sering muncul di soal LKS
□ P0	Responsive Design	Wajib untuk semua soal
□ P1	Authentication	Sering di soal backend
□ P1	Prepared Statements	Keamanan database
□ P2	Deployment	Opsional, poin bonus
□ P2	Accessibility	Opsional, differentiation

3. Sub-Kompetensi Detail per Area

HTML/CSS

- Semantic HTML5 (<header>, <main>, <section>, <article>, <nav>)
- CSS Grid layout 2D + complex layouts
- Flexbox untuk 1D alignment + distribution
- Custom properties (CSS variables) untuk theming
- Media queries + responsive breakpoints
- CSS animations & transitions untuk micro-interactions
- BEM methodology untuk class naming

JavaScript

- DOM manipulation: `querySelector`, `createElement`, event delegation
- ES6+: arrow functions, destructuring, spread, template literals
- Async JS: `fetch` API, `async/await`, `Promise`, error handling
- Form validation: constraint validation API, custom validation
- `localStorage` / `sessionStorage` untuk client-side data
- Module pattern: `import/export`

Backend (PHP / Node.js)

- Routing: parameterized routes, 404 handling
- Middleware: auth check, logging, CORS
- Form handling: POST data, file upload, validation
- Session management: login/logout, session flash messages
- REST API: GET/POST/PUT/DELETE endpoints, JSON response
- Error handling: try-catch, error pages, logging

Database Query

- SELECT with JOIN (INNER, LEFT, RIGHT)
- GROUP BY + HAVING + aggregate functions
- Subquery (correlated and non-correlated)
- Indexing untuk performance
- Prepared statement untuk SQL injection prevention
- Transaction: BEGIN, COMMIT, ROLLBACK

Deployment

- Environment variable untuk konfigurasi
- `.gitignore` untuk file sensitif
- Database migration script
- File permission basics (Linux)
- Domain + subdomain setup

4. Software & Tools LKS

Software	Versi Umum	Fungsi
VS Code	Latest	Code editor utama
XAMPP	8.x	Apache + MySQL + PHP lokal
MySQL / MariaDB	8.x / 10.x	Database server

Software	Versi Umum	Fungsi
phpMyAdmin	Latest	DB visual management
draw.io	Web/Desktop	ERD & diagram
Git	2.x	Version control
Google Chrome / Firefox	Latest	Testing + DevTools
Postman	Latest	API testing
Node.js	18+ LTS	Runtime (untuk Express)

Catatan: Saat lomba, software sudah terinstall di komputer peserta. Pastikan familiar dengan shortcut dan fitur masing-masing.

5. Latihan: Mapping & Gap Analysis

Latihan 1: Identifikasi Gap

Buka proyek web yang pernah kamu buat sebelumnya (dari modul 4, 6, 11, atau 46). Analisis:

1. Dari 4 area LKS (Web Design, Programming, Database, Admin), area mana yang paling kuat dari proyek kamu? Mengapa?
2. Area mana yang paling lemah? Berikan contoh spesifik.
3. Dari sub-kompetensi di tabel gap matrix di atas, berapa banyak yang sudah kamu kuasai? (hitung dengan jujur)
4. Buat rencana 2 minggu untuk menutup 3 gap terbesar kamu.

Latihan 2: Scoring Criteria Analysis

Ambil 1 soal dari mock test di sesi 03. Sebelum mengerjakan:

1. Baca rubrik penilaian. Tulis ulang dengan bahasamu sendiri.
2. Identifikasi 5 hal yang paling menentukan skor tinggi.
3. Berapa waktu ideal untuk tiap bagian? (frontend, backend, database)
4. Fitur apa yang bisa kamu skip jika waktu mepet?

Latihan 3: Tool Familiarity

Timer: 15 menit

1. Buka VS Code. Praktikkan shortcut:
 - Ctrl+P (Quick Open)
 - Ctrl+Shift+P (Command Palette)
 - Ctrl+D (Select next match)
 - Ctrl+/ (Toggle comment)
 - Alt+Up/Down (Move line)
2. Buka Chrome DevTools:
 - Elements tab: inspect dan edit HTML/CSS live
 - Console tab: test JS code
 - Network tab: lihat request/response
 - Application tab: lihat localStorage + cookies
3. Buka phpMyAdmin / MySQL Workbench:
 - Buat database baru
 - Jalankan query SELECT dengan JOIN
 - Export database ke SQL file

Latihan 4: Dokumentasi Gap

Buat file gap-report.md di folder proyek kamu:

```
# Gap Report – [Nama Kamu]

## Kekuatan
- [ ] Web Design – [nilai 1-5]
- [ ] Web Programming – [nilai 1-5]
- [ ] Database – [nilai 1-5]
- [ ] System Admin – [nilai 1-5]

## Top 3 Gap
1. ...
2. ...
3. ...

## Rencana Aksi (2 Minggu)
| Minggu | Target | Metode Belajar |
|-----|-----|-----|
| 1 | ... | ... |
| 2 | ... | ... |

## Catatan
...
```

6. Checklist Sesi 01

- ☐ Paham 4 area standar LKS & sub-kompetensinya
- ☐ Sudah mapping modul yang dipelajari ke standar LKS
- ☐ Tahu gap kompetensi prioritas (P0, P1, P2)
- ☐ Familiar dengan software LKS
- ☐ Selesai minimal 2 latihan mapping
- ☐ Membuat gap-report.md pribadi

Next → Sesi 02: LKS Competition Strategy

□ Sesi 02 — LKS Competition Strategy

Durasi: 120 menit

Tujuan: Menguasai strategi mengerjakan soal LKS — time management, membaca soal, scoring criteria, dan simulasi mock test

1. Time Management

Alokasi Waktu Umum (6 Jam)

Sesi	Waktu	Aktivitas	Catatan
Pembukaan	15 menit	Baca soal, rencanakan strategi	Jangan langsung coding!
Frontend	90 menit	HTML struktur, CSS styling, responsive	Prioritaskan layout
Backend	120 menit	Logic, routing, API, auth	Bagian terberat
Database	60 menit	Schema, query, CRUD	Bisa sambil backend
Testing	45 menit	Uji semua fitur, debug	Jangan skip!
Finalisasi	30 menit	Polish UI, code cleanup, docs	Nilai tambahan
Total	360 menit		

Prioritasi Berdasarkan Bobot

Bobot penilaian → alokasi waktu ideal:

Functional Correctness (35%) → ~126 menit
 Code Quality (20%) → ~72 menit
 Efficiency (15%) → ~54 menit
 UI/UX (15%) → ~54 menit
 Creativity (10%) → ~36 menit
 Documentation (5%) → ~18 menit

Strategi If-Then

Situasi	Tindakan
Waktu tinggal 2 jam, backend belum selesai	Prioritaskan core CRUD, skip fitur bonus
Waktu tinggal 1 jam, UI masih jelek	Fokus ke functional correctness, UI minimal rapi
Waktu tinggal 30 menit	Pastikan semua fitur utama jalan, jangan nambah fitur baru
Ada fitur yang mentok	Skip, lanjut ke fitur lain, balik kalau ada waktu
Error aneh	Catat, lanjut dulu, debugging terakhir

Time Boxing Technique

Gunakan timer di HP atau browser:

- ☐ 15:00 – 15:15 Baca soal & planning
- ☐ 15:15 – 16:45 Frontend (layout + responsive)
- ☐ 16:45 – 18:45 Backend (routing + logic)
- ☐ 18:45 – 19:45 Database (schema + queries)
- ☐ 19:45 – 20:30 Testing & debugging
- ☐ 20:30 – 21:00 Final polish

Tips: Set alarm tiap pergantian sesi. Jangan over-invest di satu bagian.

2. Reading Comprehension

Cara Membaca Soal LKS

Soal LKS biasanya panjang (3-5 halaman). Teknik membaca efektif:

Langkah 1: Scan Cepat (5 menit)

- Baca judul, bullet points, bold text

- Lingkari keywords: "wajib", "minimal", "tidak boleh"
- Identifikasi jumlah halaman/output yang diminta

Langkah 2: Baca Detail Per Requirement (10 menit)

- Tiap requirement → checklist
- Catat tipe data, format output, nama file

Langkah 3: Buat Mind Map (5 menit)

- Gambar arsitektur: frontend → backend → database
- Tandai dependency antar fitur

Keywords Penting

Keyword	Arti
Wajib / Must / Required	Nilai 0 kalau tidak ada
Minimal / At least	Boleh lebih, jangan kurang
Tidak boleh / Must not	Deduction point kalau dilanggar
Disarankan / Recommended	Bonus point
Jika waktu memungkinkan	Opsional, kerjakan setelah semua wajib selesai
Responsive	Harus pakai media queries
Dynamic	Data dari database / API, bukan hardcode
Validation	Client-side + server-side validation

Checklist Baca Soal

Sebelum mulai coding, pastikan:

- ☐ Ada berapa halaman/webpage yang diminta?
- ☐ Framework/library apa yang boleh/tidak boleh?
- ☐ Database: berapa tabel? relasi apa?
- ☐ Authentication: session atau JWT?
- ☐ File output: ekstensi apa? naming convention?
- ☐ Ada sample data atau harus generate sendiri?
- ☐ Waktu deadline: kapan harus submit?

3. Scoring Criteria

Rubrik Detail

Functional Correctness (35 poin)

Level	Poin	Kriteria
Excellent	30-35	Semua fitur berfungsi, edge cases tertangani
Good	20-29	Mayoritas fitur jalan, minor bug
Fair	10-19	Hanya core fitur jalan
Poor	0-9	Hanya beberapa fitur

Code Quality (20 poin)

Level	Poin	Kriteria
Excellent	18-20	Clean code, proper naming, comments, modular
Good	14-17	Mostly clean, minor inconsistency
Fair	8-13	Messy tapi masih readable
Poor	0-7	Spaghetti code, no comments

Efficiency (15 poin)

Level	Poin	Kriteria
Excellent	13-15	Optimized queries, minimal DB calls, caching
Good	10-12	Reasonable performance, minor N+1
Fair	5-9	Slow query, multiple unnecessary calls
Poor	0-4	Very slow, blocking operations

UI/UX (15 poin)

Level	Poin	Kriteria
Excellent	13-15	Responsive, consistent, accessible, polished
Good	10-12	Mostly responsive, decent aesthetics
Fair	5-9	Functional but ugly, inconsistent
Poor	0-4	Broken layout, unreadable

Creativity (10 poin)

Level	Poin	Kriteria
Excellent	9-10	Extra features, innovative solution
Good	6-8	Some additions beyond spec
Fair	3-5	Minimal extra effort
Poor	0-2	Exactly spec, nothing more

Documentation (5 poin)

Level	Poin	Kriteria
Excellent	5	README, comments, setup guide
Good	3-4	Partial docs
Fair	1-2	Minimal comments
Poor	0	No documentation

4. Common Pitfalls

❑ Pitfall 1: Missing Requirements

Masalah: Peserta melewati requirement yang tertulis jelas di soal.

Solusi: Checklist physical. Tandai tiap requirement yang sudah selesai.

❑ Pitfall 2: Hardcode Data

Masalah: Data ditulis langsung di kode, bukan dari database/API. **Solusi:** Soal biasanya bilang "dynamic". Pastikan semua konten dari DB.

❑ Pitfall 3: Ignoring Error Handling

Masalah: Tidak ada try-catch, user liat error mentah. **Solusi:** Tangani error dengan user-friendly message.

❑ Pitfall 4: No Validation

Masalah: Input user tidak divalidasi, bisa inject SQL/XSS. **Solusi:** Validate client-side (UX) + server-side (security).

❑ **Pitfall 5: Bad Time Management**

Masalah: Terlalu lama di satu fitur, fitur lain tidak selesai. **Solusi:** Time boxing. Prioritaskan fitur wajib.

❑ **Pitfall 6: Not Testing**

Masalah: Submit tanpa testing, ternyata error. **Solusi:** Sisihkan 45 menit terakhir untuk testing.

❑ **Pitfall 7: Over-Engineering**

Masalah: Bikin arsitektur terlalu kompleks untuk soal sederhana. **Solusi:** Keep it simple. YAGNI (You Ain't Gonna Need It).

❑ **Pitfall 8: Ignoring Responsive**

Masalah: Cuma desktop view, mobile broken. **Solusi:** Mobile-first CSS. Test di viewport kecil.

❑ **Pitfall 9: Forget File Naming**

Masalah: Nama file tidak sesuai spec → tidak ketemu juri. **Solusi:** Ikuti persis naming convention di soal.

❑ **Pitfall 10: Panic**

Masalah: Waktu mepet → panik → error makin banyak. **Solusi:** Tarik napas. Prioritaskan core. Sesuatu > sempurna.

Checklist Common Errors

Sebelum submit, cek:

- ☐ Semua link navigasi jalan?
- ☐ Form submit → data masuk database?
- ☐ Halaman error tidak muncul? (404, 500)
- ☐ Responsive di mobile view?
- ☐ Ada data dummy yang cukup?
- ☐ File name sesuai spec?
- ☐ Tidak ada hardcode test data?
- ☐ Session/logout jalan?
- ☐ CSS tidak broken di browser lain?
- ☐ Documentation ada?

5. Mock Test Simulation

Format Mock Test

Aspek	Detail
Durasi	6 jam (360 menit)
Soal	1 soal komprehensif (frontend + backend + database)
Tools	VS Code, XAMPP, MySQL, Chrome DevTools
Submit	ZIP folder atau upload ke server

Aturan Mock Test

1. **No internet browsing** — hanya akses dokumentasi lokal
2. **No AI tools** — kerjakan manual
3. **Timer ketat** — stop saat waktu habis
4. **Real condition** — nyalakan musik white noise, gangguan simulasi
5. **Debrief setelah selesai** — analisis apa yang bisa diperbaiki

Langkah Mock Test

flowchart TD

```
A[Baca Soal - 15min] --> B[Planning & ERD - 15min]
B --> C[Frontend Layout - 60min]
C --> D[Database Schema - 45min]
D --> E[Backend Logic - 90min]
E --> F[Integrasi FE + BE - 45min]
F --> G[Testing & Debug - 45min]
G --> H[Polish & Docs - 45min]
H --> I[Submit]
```

Self-Assessment Setelah Mock Test

Setelah selesai mock test, isi:

Mock Test Debrief

Technical

- Fitur selesai: ____ / ____ (berapa dari total)
- Fitur yang terlewat: ____
- Bug yang ditemukan: ____
- Skor estimasi:
 - Functional: ____/35
 - Code Quality: ____/20
 - Efficiency: ____/15

- UI/UX: ____/15
- Creativity: ____/10
- Documentation: ____/5
- ****Total: ____/100****

Non-Technical

- Waktu terbuang di bagian mana? ____
- Kenapa? ____
- Strategi alternatif untuk next mock: ____
- Skor manajemen waktu: ____/10
- Skor reading comprehension: ____/10

Action Items

1. ...
 2. ...
 3. ...
-

6. Latihan

Latihan 1: Time Management Simulation

Timer: 15 menit

Baca soal berikut (ringkasan):

Buat web profil perusahaan. HTML/CSS responsive. Minimal 3 halaman (home, about, contact). Form kontak dengan validasi. Data disimpan di localStorage. Tampilkan daftar pesan di halaman admin.

1. Buat breakdown tugas dalam 15 menit.
Apa yang akan kamu kerjakan duluan?
2. Berapa menit alokasi untuk tiap tugas?
3. Fitur apa yang akan kamu prioritaskan?
4. Apa yang akan kamu skip jika waktu mepet?

Latihan 2: Scoring Drill

Ambil jawaban teman (atau kode dari internet). Evaluasi:

1. Functional Correctness: fitur apa yang kurang? Berapa skor?
2. Code Quality: apakah ada hardcode? Duplikasi kode?

3. Efficiency: ada query N+1? Load data berlebihan?
4. UI/UX: responsive? Konsisten? Accessible?

Latihan 3: Mock Test Mini

Durasi: 60 menit

Buat halaman TODO App sederhana:

Fitur:

- Tambah todo (input + button)
- Tampilkan daftar todo
- Hapus todo (tombol delete)
- Todo tersimpan di localStorage
- Responsive (mobile + desktop)

Evaluasi diri setelah selesai:

- Apakah semua fitur jalan? (Y/T)
- Apakah responsif? (Y/T)
- Ada error handling? (Y/T)
- Code bersih? (Y/T)
- Selesai dalam 60 menit? (Y/T)

Latihan 4: Common Pitfalls Hunt

Baca kode berikut. Temukan minimal 5 pitfalls:

```
// app.js
const data = [
  {id: 1, name: "Adi"},
  {id: 2, name: "Budi"},
]

function getData() {
  document.getElementById("output").innerHTML = data[0].name
}

function saveData(name) {
  data.push({id: data.length+1, name})
  localStorage.setItem("data", JSON.stringify(data))
}

// Halaman admin – cuma buat admin
const isAdmin = true;
if (isAdmin) {
```

```

    // show admin panel
}

// Query
const id = req.query.id;
db.query(`SELECT * FROM users WHERE id = ${id}`)

```

Jawaban Pitfalls: 1. _____ 2. _____ 3. _____ 4. _____ 5. _____

7. Checklist Sesi 02

- ☐ Paham alokasi waktu ideal per section (6 jam)
- ☐ Bisa baca soal dengan teliti dan identifikasi keywords
- ☐ Tahu bobot penilaian dan cara memaksimalkan skor
- ☐ Hafal 10 common pitfalls dan cara menghindarinya
- ☐ Selesai mock test mini (Latihan 3)
- ☐ Sudah melakukan self-assessment

Next → Sesi 03: Mock Projects

□ Sesi 03 — LKS Mock Projects

Durasi: 240 menit (4 jam per soal) atau 6 jam untuk full mock

Tujuan: Mengerjakan soal simulasi LKS lengkap dengan answer key

□ Daftar Soal

Soal	Topik	Teknologi	Durasi	Tingkat Kesulitan
1	Web Profil Perusahaan	HTML/CSS/JS	4 jam	Medium
2	Sistem Perpustakaan	PHP/JS/MySQL	4 jam	Hard
3	Aplikasi Catatan	Express/SQLite	4 jam	Hard

Soal 01: Web Profil Perusahaan

□ Spesifikasi

Tugas

Buat website profil perusahaan **PT. Techno Nusantara** — sebuah perusahaan teknologi fiktif.

Halaman yang Diminta

1. Halaman Home (`index.html`)

- Hero section dengan banner perusahaan (gunakan gambar placeholder)
- Visi & Misi (3 bullet points)
- 3 layanan unggulan dengan card layout (icon + judul + deskripsi singkat)
- Testimonial slider (minimal 3 testimonial, auto-slide tiap 5 detik)
- Footer: copyright, social media links (placeholder)

2. Halaman About (`about.html`)

- Timeline perusahaan (minimal 4 milestone)
- Team section (4 anggota dengan foto placeholder, nama, jabatan)
- Company stats: clients, projects, awards, employees (animated counter)

3. Halaman Contact (`contact.html`)

- Form kontak: nama (required), email (required, valid), telepon, pesan (required, min 10 karakter)
- Validasi client-side dengan error message
- Simpan data pesan ke `localStorage`
- Tampilkan daftar pesan yang sudah dikirim di bawah form

4. Halaman Admin (`admin.html`)

- Login sederhana dengan username & password (hardcode: `admin/admin123`)
- Tampilkan daftar pesan dari `localStorage`
- Tombol delete per pesan
- Tombol logout

Teknis

Kriteria	Spesifikasi
CSS	Flexbox untuk layout, CSS Grid untuk card section
Responsive	Mobile-first, breakpoint 768px dan 1024px
Semantic HTML	<header>, <nav>, <main>, <section>, <footer>
JavaScript Data	Modular (pisahkan file per fungsi) localStorage untuk penyimpanan pesan
Error Handling	Validasi form, fallback gambar, error message user-friendly

Bonus (Opsional)

- Dark mode toggle
- Animasi scroll reveal
- Download data pesan sebagai CSV

□ Test Cases

TC-01: Home Page

Test	Expected
Hero section visible dengan heading	□
Card layanan 3 buah, responsive (1 kolom mobile, 3 kolom desktop)	□
Testimonial auto-slide tiap 5 detik	□
Footer selalu di bawah (sticky footer)	□

TC-02: About Page

Test	Expected
Timeline 4+ milestone tampil vertikal	☐
Team card 4 buah dengan foto placeholder	☐
Animated counter jalan saat di-scroll	☐

TC-03: Contact Form

Test	Expected
Form submit → validasi jalan	☐
Nama kosong → error message	☐
Email invalid → error message	☐
Pesan < 10 karakter → error message	☐
Form valid → data tersimpan di localStorage	☐
Data tampil di bawah form setelah submit	☐

TC-04: Admin Page

Test	Expected
Login dengan admin/admin123 → masuk	☐
Login dengan salah → error	☐
Daftar pesan tampil dari localStorage	☐
Delete pesan → pesan hilang dari localStorage	☐
Logout → kembali ke login	☐

TC-05: Responsive

Test	Expected
Viewport < 768px → layout 1 kolom	☐
Viewport 768-1024px → layout 2 kolom	☐
Viewport > 1024px → layout 3 kolom pada card	☐
Navbar mobile dengan hamburger menu	☐

☐ Scoring Checklist (100 poin)

Kategori	Item	Poin
HTML Structure (15)	Semantic HTML, halaman lengkap	15
CSS (20)	Flexbox/Grid, responsive, konsisten	20
JavaScript (25)	Form validation, localStorage, testimonial slider, counter	25
Admin (15)	Login, CRUD pesan, logout	15
UI/UX (15)	Rapi, readable, accessible, responsive	15
Code Quality (10)	Bersih, modular, comments	10
Total		100

□ Reference Answer

index.html

```

<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>PT Techno Nusantara</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header>
    <nav class="navbar">
      <div class="logo">TechnoNusa</div>
      <button class="hamburger" aria-label="Menu"></button>
      <ul class="nav-links">
        <li><a href="index.html" class="active">Home</a></li>
        <li><a href="about.html">About</a></li>
        <li><a href="contact.html">Contact</a></li>
        <li><a href="admin.html">Admin</a></li>
      </ul>
    </nav>
  </header>

```

```

        </nav>
    </header>

    <main>
        <section class="hero">
            <div class="hero-content">
                <h1>Inovasi Digital untuk Masa Depan</h1>
                <p>PT Techno Nusantara – Mitra Transformasi Digital Anda</p>
                <a href="about.html" class="btn-primary">Pelajari Lebih Lanjut</a>
            </div>
        </section>

        <section class="vision-mission">
            <h2>Visi & Misi</h2>
            <div class="vm-grid">
                <div class="vm-card">
                    <h3>Visi</h3>
                    <p>Menjadi perusahaan teknologi terdepan di Indonesia tahun
                </p>
                </div>
                <div class="vm-card">
                    <h3>Misi 1</h3>
                    <p>Menyediakan solusi digital inovatif untuk bisnis.</p>
                </div>
                <div class="vm-card">
                    <h3>Misi 2</h3>
                    <p>Membangun SDM teknologi berkualitas tinggi.</p>
                </div>
                <div class="vm-card">
                    <h3>Misi 3</h3>
                    <p>Berkontribusi pada ekosistem teknologi nasional.</p>
                </div>
            </div>
        </section>

        <section class="services">
            <h2>Layanan Kami</h2>
            <div class="services-grid">
                <div class="service-card">
                    <div class="service-icon"></div>
                    <h3>Web Development</h3>
                    <p>Aplikasi web custom dengan teknologi modern.</p>
                </div>
                <div class="service-card">
                    <div class="service-icon"></div>
                    <h3>Mobile Apps</h3>
                    <p>Aplikasi mobile Android & iOS.</p>
                </div>
            </div>
        </section>
    </main>

```

```

        </div>
        <div class="service-card">
            <div class="service-icon">▲</div>
            <h3>Cloud Solution</h3>
            <p>Infrastruktur cloud yang scalable & aman.</p>
        </div>
    </div>
</section>

<section class="testimonials">
    <h2>Testimonial</h2>
    <div class="testimonial-slider" id="testimonialSlider">
        <div class="testimonial-slide active">
            <p>"Sangat profesional! Project selesai tepat waktu."</p>
            <span class="testimonial-author">— Andi, CEO StartupX</span>
        </div>
        <div class="testimonial-slide">
            <p>"Tim teknisnya sangat responsif dan solutif."</p>
            <span class="testimonial-author">— Sari, CTO TechCorp</span>
        </div>
        <div class="testimonial-slide">
            <p>"Kualitas kode di atas ekspektasi. Recommended!"</p>
            <span class="testimonial-author">— Budi, Founder DevHouse</span>
        </div>
    </div>
</section>
</main>

<footer>
    <p>&copy; 2024 PT Techno Nusantara. All rights reserved.</p>
    <div class="social-links">
        <a href="#" aria-label="Instagram">📷</a>
        <a href="#" aria-label="LinkedIn">🌐</a>
        <a href="#" aria-label="GitHub">🐙</a>
    </div>
</footer>

<script src="js/navbar.js"></script>
<script src="js/testimonial.js"></script>
</body>
</html>

```

css/style.css

```
/* ===== RESET & BASE ===== */
*, *::before, *::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  line-height: 1.6;
  color: #333;
  min-height: 100vh;
  display: flex;
  flex-direction: column;
}

main {
  flex: 1;
}

/* ===== NAVBAR ===== */
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1rem 5%;
  background: #2c3e50;
  color: white;
}

.logo {
  font-size: 1.5rem;
  font-weight: bold;
}

.nav-links {
  display: flex;
  list-style: none;
  gap: 2rem;
}

.nav-links a {
  color: white;
  text-decoration: none;
}
```

```

        transition: color 0.3s;
    }

    .nav-links a:hover,
    .nav-links a.active {
        color: #3498db;
    }

    .hamburger {
        display: none;
        background: none;
        border: none;
        color: white;
        font-size: 1.5rem;
        cursor: pointer;
    }

    /* ===== HERO ===== */
    .hero {
        background: linear-gradient(135deg, #3498db, #2c3e50);
        color: white;
        text-align: center;
        padding: 4rem 1rem;
    }

    .hero h1 {
        font-size: 2.5rem;
        margin-bottom: 1rem;
    }

    .btn-primary {
        display: inline-block;
        background: #e74c3c;
        color: white;
        padding: 0.8rem 2rem;
        text-decoration: none;
        border-radius: 5px;
        margin-top: 1rem;
        transition: background 0.3s;
    }

    .btn-primary:hover {
        background: #c0392b;
    }

    /* ===== VISION MISSION ===== */

```

```

.vision-mission,
.services,
.testimonials {
    padding: 3rem 5%;
}

.vision-mission h2,
.services h2,
.testimonials h2 {
    text-align: center;
    margin-bottom: 2rem;
    font-size: 2rem;
    color: #2c3e50;
}

.vm-grid,
.services-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
    gap: 2rem;
}

.vm-card,
.service-card {
    background: #f8f9fa;
    padding: 2rem;
    border-radius: 10px;
    text-align: center;
    box-shadow: 0 2px 10px rgba(0,0,0,0.1);
}

/* ===== TESTIMONIALS ===== */
.testimonial-slider {
    position: relative;
    max-width: 600px;
    margin: 0 auto;
    overflow: hidden;
}

.testimonial-slide {
    display: none;
    text-align: center;
    padding: 2rem;
    background: #f8f9fa;
    border-radius: 10px;
}

```

```

.testimonial-slide.active {
    display: block;
    animation: fadeIn 0.5s;
}

@keyframes fadeIn {
    from { opacity: 0; }
    to { opacity: 1; }
}

.testimonial-author {
    display: block;
    margin-top: 1rem;
    font-style: italic;
    color: #7f8c8d;
}

/* ===== FOOTER ===== */
footer {
    background: #2c3e50;
    color: white;
    text-align: center;
    padding: 1.5rem;
    margin-top: auto;
}

.social-links {
    margin-top: 0.5rem;
}

.social-links a {
    color: white;
    text-decoration: none;
    margin: 0 0.5rem;
    font-size: 1.2rem;
}

/* ===== RESPONSIVE ===== */
@media (max-width: 768px) {
    .nav-links {
        display: none;
        flex-direction: column;
        position: absolute;
        top: 60px;
        left: 0;
    }
}

```

```

        width: 100%;
        background: #2c3e50;
        padding: 1rem;
    }

    .nav-links.show {
        display: flex;
    }

    .hamburger {
        display: block;
    }

    .hero h1 {
        font-size: 1.8rem;
    }

    .vm-grid,
    .services-grid {
        grid-template-columns: 1fr;
    }
}

@media (min-width: 769px) and (max-width: 1024px) {
    .services-grid {
        grid-template-columns: repeat(2, 1fr);
    }
}

```

js/navbar.js

```

document.addEventListener('DOMContentLoaded', function() {
    const hamburger = document.querySelector('.hamburger');
    const navLinks = document.querySelector('.nav-links');

    hamburger.addEventListener('click', function() {
        navLinks.classList.toggle('show');
    });

    // Close menu when link clicked
    document.querySelectorAll('.nav-links a').forEach(link => {
        link.addEventListener('click', () => {
            navLinks.classList.remove('show');
        });
    });
});

```

```
});
```

js/testimonial.js

```
document.addEventListener('DOMContentLoaded', function() {  
  const slides = document.querySelectorAll('.testimonial-slide');  
  let currentSlide = 0;  
  
  function showSlide(index) {  
    slides.forEach(slide => slide.classList.remove('active'));  
    slides[index].classList.add('active');  
  }  
  
  function nextSlide() {  
    currentSlide = (currentSlide + 1) % slides.length;  
    showSlide(currentSlide);  
  }  
  
  if (slides.length > 0) {  
    showSlide(0);  
    setInterval(nextSlide, 5000);  
  }  
});
```

js/form.js

```
document.addEventListener('DOMContentLoaded', function() {  
  const form = document.getElementById('contactForm');  
  if (!form) return;  
  
  const messagesContainer = document.getElementById('messagesList');  
  
  // Load existing messages  
  loadMessages();  
  
  form.addEventListener('submit', function(e) {  
    e.preventDefault();  
  
    // Reset errors  
    document.querySelectorAll('.error').forEach(el => el.remove());  
  
    // Get values  
    const name = document.getElementById('name').value.trim();  
    const email = document.getElementById('email').value.trim();  
    const phone = document.getElementById('phone').value.trim();
```

```

const message = document.getElementById('message').value.trim();

let isValid = true;

// Validation
if (!name) {
  showError('name', 'Nama harus diisi');
  isValid = false;
}

if (!email) {
  showError('email', 'Email harus diisi');
  isValid = false;
} else if (!isValidEmail(email)) {
  showError('email', 'Format email tidak valid');
  isValid = false;
}

if (!message) {
  showError('message', 'Pesan harus diisi');
  isValid = false;
} else if (message.length < 10) {
  showError('message', 'Pesan minimal 10 karakter');
  isValid = false;
}

if (!isValid) return;

// Save message
const messages = JSON.parse(localStorage.getItem('contacts') || '[]');
messages.push({
  id: Date.now(),
  name,
  email,
  phone,
  message,
  date: new Date().toISOString()
});
localStorage.setItem('contacts', JSON.stringify(messages));

// Clear form
form.reset();

// Reload messages
loadMessages();

```

```

        alert('Pesan berhasil dikirim!');
    });

    function showError(fieldId, msg) {
        const field = document.getElementById(fieldId);
        const error = document.createElement('div');
        error.className = 'error';
        error.style.color = 'red';
        error.style.fontSize = '0.85rem';
        error.textContent = msg;
        field.parentNode.insertBefore(error, field.nextSibling);
    }

    function isValidEmail(email) {
        return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
    }

    function loadMessages() {
        if (!messagesContainer) return;
        const messages = JSON.parse(localStorage.getItem('contacts') || '[]');

        if (messages.length === 0) {
            messagesContainer.innerHTML = '<p>Belum ada pesan.</p>';
            return;
        }

        messagesContainer.innerHTML = messages.map(msg => `
            <div class="message-item">
                <strong>${escapeHtml(msg.name)}</strong> (${escapeHtml(msg.email)}
                <p>${escapeHtml(msg.message)}</p>
                <small>${new Date(msg.date).toLocaleString()}</small>
            </div>
        `).join('');
    }

    function escapeHtml(str) {
        const div = document.createElement('div');
        div.textContent = str;
        return div.innerHTML;
    }
});

```

js/admin.js

```
document.addEventListener('DOMContentLoaded', function() {
  // Check login status
  const isLoggedIn = sessionStorage.getItem('adminLoggedIn');
  const loginForm = document.getElementById('loginForm');
  const adminContent = document.getElementById('adminContent');

  if (isLoggedIn === 'true') {
    showAdmin();
  }

  if (loginForm) {
    loginForm.addEventListener('submit', function(e) {
      e.preventDefault();
      const username = document.getElementById('username').value;
      const password = document.getElementById('password').value;

      if (username === 'admin' && password === 'admin123') {
        sessionStorage.setItem('adminLoggedIn', 'true');
        showAdmin();
      } else {
        alert('Username atau password salah!');
      }
    });
  }

  function showAdmin() {
    if (loginForm) loginForm.style.display = 'none';
    if (adminContent) {
      adminContent.style.display = 'block';
      loadAdminMessages();
    }
  }

  function loadAdminMessages() {
    const list = document.getElementById('adminMessagesList');
    const messages = JSON.parse(localStorage.getItem('contacts') || '[]');

    if (messages.length === 0) {
      list.innerHTML = '<p>Tidak ada pesan.</p>';
      return;
    }

    list.innerHTML = messages.map(msg => `
      <div class="admin-message">
```

```

        <p><strong>${escapeHtml(msg.name)}</strong> – ${escapeHtml(msg.e
        <p>${escapeHtml(msg.message)}</p>
        <small>${new Date(msg.date).toLocaleString()}</small>
        <button onclick="deleteMessage(${msg.id})" class="btn-delete">Ha
    </div>
    `).join('');
}

window.deleteMessage = function(id) {
    let messages = JSON.parse(localStorage.getItem('contacts') || '[]');
    messages = messages.filter(msg => msg.id !== id);
    localStorage.setItem('contacts', JSON.stringify(messages));
    loadAdminMessages();
};

window.logout = function() {
    sessionStorage.removeItem('adminLoggedIn');
    location.reload();
};

function escapeHtml(str) {
    const div = document.createElement('div');
    div.textContent = str;
    return div.innerHTML;
}
});

```

Soal 02: Sistem Perpustakaan

□ Spesifikasi

Tugas

Buat sistem manajemen perpustakaan sederhana dengan PHP + MySQL.

Requirements

Database

- Nama database: library_db
- Tabel books:
 - id (INT, AUTO_INCREMENT, PRIMARY KEY)
 - title (VARCHAR 255, NOT NULL)

- author (VARCHAR 150, NOT NULL)
- isbn (VARCHAR 20, UNIQUE)
- category (VARCHAR 100)
- year (YEAR)
- stock (INT, DEFAULT 1)
- created_at (TIMESTAMP)
- Tabel members:
 - id (INT, AUTO_INCREMENT, PRIMARY KEY)
 - name (VARCHAR 150, NOT NULL)
 - email (VARCHAR 150, UNIQUE)
 - phone (VARCHAR 20)
 - join_date (DATE)
- Tabel loans:
 - id (INT, AUTO_INCREMENT, PRIMARY KEY)
 - book_id (INT, FK → books)
 - member_id (INT, FK → members)
 - loan_date (DATE)
 - return_date (DATE, NULLABLE)
 - status (ENUM: 'active', 'returned')

Halaman & Fitur

1. Login (login.php)

- Session-based authentication
- Username/password hardcoded: admin / admin123
- Redirect ke dashboard jika sudah login

2. Dashboard (index.php)

- Summary cards: total buku, total member, buku dipinjam, buku tersedia
- Query aggregate dari database

3. Manajemen Buku (books.php)

- Tabel daftar buku dengan pagination (10 per halaman)
- Search by title atau author (AJAX, real-time)
- Form tambah buku dengan validasi
- Edit buku (modal atau halaman terpisah)
- Hapus buku dengan konfirmasi

4. Manajemen Member (members.php)

- Tabel daftar member dengan pagination

- Search by name or email
- CRUD member

5. Transaksi Peminjaman (loans.php)

- Pinjam buku: pilih buku + member via dropdown
- Kembalikan buku: klik tombol return
- Tampilkan status (active/returned)
- Validasi: buku yang sudah dipinjam tidak bisa dipinjam lagi sebelum dikembalikan

6. Laporan (report.php)

- Grafik/summary: buku terpopuler (most borrowed)
- Daftar member dengan pinjaman terbanyak
- Export data ke format HTML table (bisa dicopy)

Teknis

Kriteria	Spesifikasi
PHP	Procedural atau OOP (bebas), prepared statements
MySQL	Minimal 3 tabel dengan relasi
CSS	Bootstrap 5 (CDN) atau custom
JS	AJAX untuk search, fetch API optional
Security	Prepared statements, session, XSS protection
Responsive	Bootstrap grid atau custom media queries

Bonus (Optional)

- Fine/denda keterlambatan (hitung berdasarkan tanggal kembali)
- Export to PDF
- Dark mode
- Image upload untuk sampul buku

□ Test Cases

TC-01: Login

Test	Expected
Akses dashboard tanpa login → redirect ke login	□
Login dengan admin/admin123 → masuk dashboard	□

Test	Expected
Login dengan salah → error message	☐
Logout → session destroyed	☐

TC-02: Books CRUD

Test	Expected
Tambah buku dengan data valid → tersimpan di DB	☐
Tambah buku dengan ISBN duplikat → error	☐
Edit buku → data berubah	☐
Hapus buku → data hilang dari DB	☐
Search by title → hasil filter	☐
Pagination berfungsi (10 data per halaman)	☐

TC-03: Members CRUD

Test	Expected
Tambah member valid → tersimpan	☐
Email duplikat → error	☐
Edit member → update	☐
Hapus member → data hilang	☐
Search by name → hasil filter	☐

TC-04: Loans

Test	Expected
Pinjam buku → loan record tersimpan, status 'active'	☐
Buku stok 0 → tidak bisa dipinjam	☐
Kembalikan buku → status 'returned', return_date terisi	☐
Buku yang masih active → tidak muncul di dropdown pinjam	☐

TC-05: Report

Test	Expected
Buku terpopuler tampil (diurutkan)	☐
Member aktif tampil	☐
Tabel rapi dan bisa dicopy	☐

□ Scoring Checklist (100 poin)

Kategori	Item	Poin
Database (20)	3 tabel, relasi, integrity	20
CRUD Books (20)	Create, Read, Update, Delete + search + pagination	20
CRUD Members (15)	Create, Read, Update, Delete + search	15
Loans (20)	Pinjam, kembali, validasi stok	20
Report (10)	Query agregat, tampilan rapi	10
UI/UX (10)	Bootstrap/custom, responsive, consistent	10
Security (5)	Prepared statements, session, XSS protection	5
Total		100

□ Reference Answer

config/database.php

```
<?php
$host = 'localhost';
$dbname = 'library_db';
$username = 'root';
$password = '';

try {
    $pdo = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8mb4", $username,
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    $pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);
} catch(PDOException $e) {
    die("Connection failed: " . $e->getMessage());
}
?>
```

schema.sql

```
CREATE DATABASE IF NOT EXISTS library_db;  
USE library_db;
```

```
CREATE TABLE books (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    title VARCHAR(255) NOT NULL,  
    author VARCHAR(150) NOT NULL,  
    isbn VARCHAR(20) UNIQUE,  
    category VARCHAR(100),  
    year YEAR,  
    stock INT DEFAULT 1,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE members (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(150) NOT NULL,  
    email VARCHAR(150) UNIQUE,  
    phone VARCHAR(20),  
    join_date DATE DEFAULT (CURRENT_DATE)  
);
```

```
CREATE TABLE loans (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    book_id INT NOT NULL,  
    member_id INT NOT NULL,  
    loan_date DATE DEFAULT (CURRENT_DATE),  
    return_date DATE DEFAULT NULL,  
    status ENUM('active', 'returned') DEFAULT 'active',  
    FOREIGN KEY (book_id) REFERENCES books(id) ON DELETE CASCADE,  
    FOREIGN KEY (member_id) REFERENCES members(id) ON DELETE CASCADE  
);
```

-- Sample data

```
INSERT INTO books (title, author, isbn, category, year, stock) VALUES  
( 'Pemrograman Web', 'Andi Susanto', '978-602-01-2345-6', 'Teknologi', 2023, 3),  
( 'Basis Data Lanjutan', 'Budi Pratama', '978-602-01-2346-3', 'Teknologi', 2022, 5),  
( 'Algoritma & Struktur Data', 'Citra Dewi', '978-602-01-2347-0', 'Teknologi', 2021, 10);
```

```
INSERT INTO members (name, email, phone) VALUES  
( 'Rina Amelia', 'rina@email.com', '081234567890'),  
( 'Dimas Ardianto', 'dimas@email.com', '081234567891');
```

login.php

```
<?php
session_start();
if (isset($_SESSION['admin_logged_in']) && $_SESSION['admin_logged_in'] === true) {
    header('Location: index.php');
    exit;
}

$error = '';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $username = $_POST['username'] ?? '';
    $password = $_POST['password'] ?? '';

    if ($username === 'admin' && $password === 'admin123') {
        $_SESSION['admin_logged_in'] = true;
        $_SESSION['admin_username'] = $username;
        header('Location: index.php');
        exit;
    } else {
        $error = 'Username atau password salah!';
    }
}
?>
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login – Sistem Perpustakaan</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap."
</head>
<body class="bg-light">
    <div class="container mt-5">
        <div class="row justify-content-center">
            <div class="col-md-4">
                <div class="card">
                    <div class="card-header">
                        <h3 class="text-center">Login Admin</h3>
                    </div>
                    <div class="card-body">
                        <?php if ($error): ?>
                            <div class="alert alert-danger"><?= htmlspecialchars($error)</div>
                        <?php endif; ?>
                        <form method="POST">
                            <div class="mb-3">
```



```

<div class="container mt-4">
  <h1>Dashboard</h1>
  <div class="row mt-4">
    <div class="col-md-3">
      <div class="card text-white bg-primary mb-3">
        <div class="card-body">
          <h5 class="card-title">Total Buku</h5>
          <p class="card-text display-6"><?= $totalBooks ?></p>
        </div>
      </div>
    </div>
    <div class="col-md-3">
      <div class="card text-white bg-success mb-3">
        <div class="card-body">
          <h5 class="card-title">Total Member</h5>
          <p class="card-text display-6"><?= $totalMembers ?></p>
        </div>
      </div>
    </div>
    <div class="col-md-3">
      <div class="card text-white bg-warning mb-3">
        <div class="card-body">
          <h5 class="card-title">Dipinjam</h5>
          <p class="card-text display-6"><?= $activeLoans ?></p>
        </div>
      </div>
    </div>
    <div class="col-md-3">
      <div class="card text-white bg-info mb-3">
        <div class="card-body">
          <h5 class="card-title">Tersedia</h5>
          <p class="card-text display-6"><?= $availableBooks ?></p>
        </div>
      </div>
    </div>
  </div>
</div>

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.
</body>
</html>

```

books.php

```
<?php
session_start();
if (!isset($_SESSION['admin_logged_in']) || $_SESSION['admin_logged_in'] !== true) {
    header('Location: login.php');
    exit;
}

require_once 'config/database.php';

$page = isset($_GET['page']) ? (int)$_GET['page'] : 1;
$limit = 10;
$offset = ($page - 1) * $limit;
$search = $_GET['search'] ?? '';

// Handle CRUD
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (isset($_POST['action'])) {
        // Add book
        if ($_POST['action'] === 'add') {
            $stmt = $pdo->prepare("INSERT INTO books (title, author, isbn, category, year, stock) VALUES (?, ?, ?, ?, ?, ?)");
            $stmt->execute([
                $_POST['title'],
                $_POST['author'],
                $_POST['isbn'],
                $_POST['category'],
                $_POST['year'],
                $_POST['stock'] ?? 1
            ]);
        }
        // Edit book
        elseif ($_POST['action'] === 'edit') {
            $stmt = $pdo->prepare("UPDATE books SET title=?, author=?, isbn=?, category=?, year=?, stock=? WHERE id=?");
            $stmt->execute([
                $_POST['title'],
                $_POST['author'],
                $_POST['isbn'],
                $_POST['category'],
                $_POST['year'],
                $_POST['stock'] ?? 1,
                $_POST['id']
            ]);
        }
        // Delete book
        elseif ($_POST['action'] === 'delete') {
            $stmt = $pdo->prepare("DELETE FROM books WHERE id=?");
            $stmt->execute([
                $_POST['id']
            ]);
        }
    }
}
```

```

        $stmt = $pdo->prepare("DELETE FROM books WHERE id = ?");
        $stmt->execute([$$_POST['id']]);
    }
    header('Location: books.php');
    exit;
}

// Build query
$where = '';
$params = [];
if ($search) {
    $where = "WHERE title LIKE ? OR author LIKE ?";
    $params = ["%$search%", "%$search%"];
}

$total = $pdo->prepare("SELECT COUNT(*) FROM books $where");
$total->execute($params);
$totalBooks = $total->fetchColumn();
$totalPages = ceil($totalBooks / $limit);

$stmt = $pdo->prepare("SELECT * FROM books $where ORDER BY created_at DESC LIMIT $limit");
$stmt->execute($params);
$books = $stmt->fetchAll();
?>
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Manajemen Buku – Sistem Perpustakaan</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>
    <?php include 'includes/navbar.php'; ?>

    <div class="container mt-4">
        <h1>Manajemen Buku</h1>

        <!-- Search -->
        <div class="row mb-3">
            <div class="col-md-6">
                <input type="text" id="searchInput" class="form-control" placeholder="Cari buku disini" />
            </div>
            <div class="col-md-6 text-end">
                <button class="btn btn-primary" data-bs-toggle="modal" data-bs-target="#searchModal">Cari</button>
            </div>
        </div>
    </div>

```

```

        </div>
    </div>

    <!-- Table -->
    <div class="table-responsive">
        <table class="table table-striped">
            <thead>
                <tr>
                    <th>ID</th>
                    <th>Judul</th>
                    <th>Penulis</th>
                    <th>ISBN</th>
                    <th>Kategori</th>
                    <th>Tahun</th>
                    <th>Stok</th>
                    <th>Aksi</th>
                </tr>
            </thead>
            <tbody id="booksTableBody">
                <?php foreach ($books as $book): ?>
                    <tr>
                        <td><?= $book['id'] ?></td>
                        <td><?= htmlspecialchars($book['title']) ?></td>
                        <td><?= htmlspecialchars($book['author']) ?></td>
                        <td><?= htmlspecialchars($book['isbn']) ?></td>
                        <td><?= htmlspecialchars($book['category']) ?></td>
                        <td><?= $book['year'] ?></td>
                        <td><?= $book['stock'] ?></td>
                        <td>
                            <button class="btn btn-sm btn-warning" onclick="edit
                            <form method="POST" style="display:inline" onsubmit=
                                <input type="hidden" name="action" value="delete
                                <input type="hidden" name="id" value="<?= $book[
                                <button class="btn btn-sm btn-danger">Hapus</but
                            </form>
                        </td>
                    </tr>
                <?php endforeach; ?>
                <?php if (empty($books)): ?>
                    <tr><td colspan="8" class="text-center">Tidak ada data buku.
                <?php endif; ?>
            </tbody>
        </table>
    </div>

    <!-- Pagination -->

```

```

<nav>
  <ul class="pagination">
    <?php for ($i = 1; $i <= $totalPages; $i++): ?>
      <li class="page-item <?=$i === $page ? 'active' : '' ?>">
        <a class="page-link" href="?page=<?=$i ?>&search=<?=$urlencod
      </li>
    <?php endforeach; ?>
  </ul>
</nav>
</div>

<!-- Add Book Modal -->
<div class="modal fade" id="addBookModal" tabindex="-1">
  <div class="modal-dialog">
    <div class="modal-content">
      <form method="POST">
        <div class="modal-header">
          <h5 class="modal-title">Tambah Buku</h5>
          <button type="button" class="btn-close" data-bs-dismiss=
        </div>
        <div class="modal-body">
          <input type="hidden" name="action" value="add">
          <div class="mb-3">
            <label class="form-label">Judul *</label>
            <input type="text" name="title" class="form-control"
          </div>
          <div class="mb-3">
            <label class="form-label">Penulis *</label>
            <input type="text" name="author" class="form-control"
          </div>
          <div class="mb-3">
            <label class="form-label">ISBN</label>
            <input type="text" name="isbn" class="form-control"
          </div>
          <div class="mb-3">
            <label class="form-label">Kategori</label>
            <input type="text" name="category" class="form-contr
          </div>
          <div class="mb-3">
            <label class="form-label">Tahun</label>
            <input type="number" name="year" class="form-control"
          </div>
          <div class="mb-3">
            <label class="form-label">Stok</label>
            <input type="number" name="stock" class="form-control
          </div>
        </div>
      </form>
    </div>
  </div>
</div>

```

```

        </div>
        <div class="modal-footer">
            <button type="button" class="btn btn-secondary" data-bs-
            <button type="submit" class="btn btn-primary">Simpan</bu
        </div>
    </form>
</div>
</div>
</div>

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.
<script>
// Real-time search with AJAX
document.getElementById('searchInput')?.addEventListener('keyup', function()
    const search = this.value;
    const xhr = new XMLHttpRequest();
    xhr.open('GET', `ajax_search_books.php?search=${encodeURIComponent(search)}`);
    xhr.onload = function() {
        if (xhr.status === 200) {
            document.getElementById('booksTableBody').innerHTML = xhr.response;
        }
    };
    xhr.send();
});

function editBook(id) {
    // Simplified: redirect to edit page or use another modal
    alert('Edit book ID: ' + id + ' - implement edit modal here');
}
</script>
</body>
</html>

```

ajax_search_books.php

```

<?php
require_once 'config/database.php';

$search = $_GET['search'] ?? '';
$where = '';
$params = [];
if ($search) {
    $where = "WHERE title LIKE ? OR author LIKE ?";
    $params = ["%$search%", "%$search%"];
}

```

```

$stmt = $pdo->prepare("SELECT * FROM books $where ORDER BY created_at DESC LIMIT 10");
$stmt->execute($params);
$books = $stmt->fetchAll();

foreach ($books as $book): ?>
<tr>
    <td><?= $book['id'] ?></td>
    <td><?= htmlspecialchars($book['title']) ?></td>
    <td><?= htmlspecialchars($book['author']) ?></td>
    <td><?= htmlspecialchars($book['isbn']) ?></td>
    <td><?= htmlspecialchars($book['category']) ?></td>
    <td><?= $book['year'] ?></td>
    <td><?= $book['stock'] ?></td>
    <td>
        <button class="btn btn-sm btn-warning" onclick="editBook(<?= $book['id'] ?>)">Edit</button>
        <form method="POST" style="display:inline" onsubmit="return confirm('Hapus buku ini?');">
            <input type="hidden" name="action" value="delete">
            <input type="hidden" name="id" value="<?= $book['id'] ?>">
            <button class="btn btn-sm btn-danger">Hapus</button>
        </form>
    </td>
</tr>
<?php endforeach; ?>
<?php if (empty($books)): ?>
<tr><td colspan="8" class="text-center">Tidak ada data buku.</td></tr>
<?php endif; ?>

```

includes/navbar.php

```

<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container">
        <a class="navbar-brand" href="index.php"> Perpustakaan</a>
        <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav"
            <span class="navbar-toggler-icon"></span>
        </button>
        <div class="collapse navbar-collapse" id="navbarNav">
            <ul class="navbar-nav">
                <li class="nav-item"><a class="nav-link" href="index.php">Dashboard</a>
                <li class="nav-item"><a class="nav-link" href="books.php">Buku</a>
                <li class="nav-item"><a class="nav-link" href="members.php">Anggota</a>
                <li class="nav-item"><a class="nav-link" href="loans.php">Pinjaman</a>
                <li class="nav-item"><a class="nav-link" href="report.php">Laporan</a>
            </ul>
            <ul class="navbar-nav ms-auto">

```

```

        <li class="nav-item"><a class="nav-link" href="logout.php">Logout
      </li>
    </ul>
  </div>
</div>
</nav>

```

Soal 03: Aplikasi Catatan (Notes API)

□ Spesifikasi

Tugas

Buat REST API untuk aplikasi catatan (notes) menggunakan **Express.js** + **SQLite** (atau PostgreSQL). Sertakan frontend sederhana dengan AJAX.

Backend API

Method	Endpoint	Deskripsi	Auth
POST	/api/auth/register	Register user baru	No
POST	/api/auth/login	Login, return JWT token	No
GET	/api/notes	Ambil semua catatan user	Yes
GET	/api/notes/:id	Ambil detail catatan	Yes
POST	/api/notes	Buat catatan baru	Yes
PUT	/api/notes/:id	Update catatan	Yes
DELETE	/api/notes/:id	Hapus catatan	Yes

Database Schema

```

-- Users table
CREATE TABLE users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  username VARCHAR(100) UNIQUE NOT NULL,
  email VARCHAR(150) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Notes table
CREATE TABLE notes (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL,

```

```

    title VARCHAR(255) NOT NULL,
    content TEXT,
    category VARCHAR(100) DEFAULT 'general',
    is_archived BOOLEAN DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

```

Frontend (Static Files)

Halaman Login/Register

- Tab switch antara login dan register
- Form validation client-side
- Simpan JWT token di localStorage

Halaman Notes

- Tampilkan daftar catatan (cards)
- Filter by category (dropdown)
- Search by title (real-time)
- Create note (modal)
- Edit note (modal, pre-filled)
- Delete note (konfirmasi)
- Archive / Unarchive toggle
- Logout button

Teknis

Kriteria	Spesifikasi
Node.js	Express.js, middleware untuk auth
Database	SQLite (better-sqlite3 atau sqlite3) atau PostgreSQL
Auth	JWT (jsonwebtoken), bcrypt untuk password
Frontend	HTML + CSS + vanilla JS (fetch API)
API	RESTful, JSON response, proper HTTP status codes
Validation	express-validator atau manual
Error Handling	Global error handler middleware

Bonus (Opsional)

- Pagination (page + limit query params)
- Sort by created_at / title
- Tags system (many-to-many)

- Markdown support untuk content
- Share note via public link

□ Test Cases

TC-01: Authentication

Test	Expected
Register dengan data valid → 201, return user info	□
Register dengan email duplikat → 409 error	□
Login dengan kredensial benar → 200, return JWT	□
Login dengan password salah → 401 error	□
Akses endpoint tanpa token → 401 error	□

TC-02: Notes CRUD

Test	Expected
POST /api/notes → 201, return note	□
GET /api/notes → 200, return array	□
GET /api/notes/:id → 200, return detail	□
PUT /api/notes/:id → 200, return updated	□
DELETE /api/notes/:id → 200, success message	□
Akses note user lain → 403 error	□

TC-03: Filter & Search

Test	Expected
Filter by category → hanya notes kategori tersebut	□
Search by title → notes yang match	□
Kombinasi filter + search → hasil sesuai	□

TC-04: Frontend

Test	Expected
Login form → sukses login, redirect ke notes page	□
Register form → sukses, redirect ke login	□
Create note → muncul di daftar	□

Test	Expected
Edit note → data berubah	☐
Delete note → hilang dari daftar	☐
Archive → tidak tampil di active, tampil di archived	☐
Logout → redirect ke login page	☐

☐ Scoring Checklist (100 poin)

Kategori	Item	Poin
API Design (20)	RESTful, proper HTTP codes, JSON response	20
Auth (20)	Register, login, JWT, bcrypt, protected routes	20
Notes CRUD (25)	Create, Read, Update, Delete, ownership check	25
Database (10)	Schema, relations, migrations	10
Frontend (15)	Login/register, notes list, CRUD UI, filter	15
Error Handling (5)	Global error handler, validation errors	5
Code Quality (5)	Modular, clean, comments, .env config	5
Total		100

☐ Reference Answer

package.json

```
{
  "name": "notes-api",
```

```

    "version": "1.0.0",
    "main": "server.js",
    "scripts": {
      "start": "node server.js",
      "dev": "node server.js"
    },
    "dependencies": {
      "express": "^4.18.2",
      "better-sqlite3": "^9.4.3",
      "jsonwebtoken": "^9.0.2",
      "bcryptjs": "^2.4.3",
      "cors": "^2.8.5",
      "dotenv": "^16.3.1"
    }
  }
}

```

.env

```

JWT_SECRET=rahasia_lks_2024
PORT=3000

```

server.js

```

require('dotenv').config();
const express = require('express');
const cors = require('cors');
const path = require('path');

const app = express();
const PORT = process.env.PORT || 3000;

// Middleware
app.use(cors());
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

// Database
const Database = require('better-sqlite3');
const db = new Database('notes.db');

// Initialize tables
db.exec(`
  CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT UNIQUE NOT NULL,

```

```

    email TEXT UNIQUE NOT NULL,
    password TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
  );

CREATE TABLE IF NOT EXISTS notes (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL,
  title TEXT NOT NULL,
  content TEXT,
  category TEXT DEFAULT 'general',
  is_archived INTEGER DEFAULT 0,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
`);

```

// Routes

```

const authRoutes = require('./routes/auth');
const notesRoutes = require('./routes/notes');

```

```

app.use('/api/auth', authRoutes(db));
app.use('/api/notes', notesRoutes(db));

```

// Global error handler

```

app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(err.status || 500).json({
    error: err.message || 'Internal server error'
  });
});

```

```

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});

```

middleware/auth.js

```

const jwt = require('jsonwebtoken');

function authenticateToken(req, res, next) {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1]; // Bearer TOKEN

```

```

    if (!token) {
      return res.status(401).json({ error: 'Token tidak ditemukan' });
    }

    jwt.verify(token, process.env.JWT_SECRET, (err, user) => {
      if (err) {
        return res.status(403).json({ error: 'Token tidak valid' });
      }
      req.user = user;
      next();
    });
  });
}

```

```

module.exports = authenticateToken;

```

routes/auth.js

```

const express = require('express');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');

```

```

module.exports = function(db) {
  const router = express.Router();

```

```

  // POST /api/auth/register

```

```

  router.post('/register', (req, res) => {

```

```

    try {

```

```

      const { username, email, password } = req.body;

```

```

      if (!username || !email || !password) {

```

```

        return res.status(400).json({ error: 'Semua field wajib diisi' });
      }

```

```

      if (password.length < 6) {

```

```

        return res.status(400).json({ error: 'Password minimal 6 karakter' });
      }

```

```

      // Check existing user

```

```

      const existing = db.prepare('SELECT id FROM users WHERE email = ? OR username = ?');

```

```

      if (existing) {

```

```

        return res.status(409).json({ error: 'Email atau username sudah ada' });
      }

```

```

      // Hash password

```

```

      const hashedPassword = bcrypt.hashSync(password, 10);

```

```

    // Insert user
    const result = db.prepare('INSERT INTO users (username, email, password) VALUES (?, ?, ?)').run(
      req.body.username, req.body.email, req.body.password
    );

    res.status(201).json({
      message: 'Registrasi berhasil',
      user: { id: result.lastInsertRowid, username, email }
    });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// POST /api/auth/login
router.post('/login', (req, res) => {
  try {
    const { email, password } = req.body;

    if (!email || !password) {
      return res.status(400).json({ error: 'Email dan password wajib diisi' });
    }

    // Find user
    const user = db.prepare('SELECT * FROM users WHERE email = ?').get(email);
    if (!user) {
      return res.status(401).json({ error: 'Email atau password salah' });
    }

    // Check password
    const validPassword = bcrypt.compareSync(password, user.password);
    if (!validPassword) {
      return res.status(401).json({ error: 'Email atau password salah' });
    }

    // Generate JWT
    const token = jwt.sign(
      { id: user.id, username: user.username, email: user.email },
      process.env.JWT_SECRET,
      { expiresIn: '24h' }
    );

    res.json({
      message: 'Login berhasil',
      token,
      user: { id: user.id, username: user.username, email: user.email }
    });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

```

```

    } catch (err) {
      res.status(500).json({ error: err.message });
    }
  });

  return router;
};

```

routes/notes.js

```

const express = require('express');
const authenticateToken = require('../middleware/auth');

module.exports = function(db) {
  const router = express.Router();

  // All routes require authentication
  router.use(authenticateToken);

  // GET /api/notes - Get all notes for current user
  router.get('/', (req, res) => {
    try {
      const { category, search, archived } = req.query;
      let query = 'SELECT * FROM notes WHERE user_id = ?';
      const params = [req.user.id];

      if (archived === 'true') {
        query += ' AND is_archived = 1';
      } else {
        query += ' AND is_archived = 0';
      }

      if (category) {
        query += ' AND category = ?';
        params.push(category);
      }

      if (search) {
        query += ' AND title LIKE ?';
        params.push(`%${search}%`);
      }

      query += ' ORDER BY updated_at DESC';

      const notes = db.prepare(query).all(...params);
    }
  });
};

```

```

        res.json(notes);
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});

// GET /api/notes/:id - Get single note
router.get('/:id', (req, res) => {
    try {
        const note = db.prepare('SELECT * FROM notes WHERE id = ? AND user_id = ?').get(req.params.id, req.user.id);

        if (!note) {
            return res.status(404).json({ error: 'Catatan tidak ditemukan' });
        }

        res.json(note);
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});

// POST /api/notes - Create note
router.post('/', (req, res) => {
    try {
        const { title, content, category } = req.body;

        if (!title) {
            return res.status(400).json({ error: 'Judul wajib diisi' });
        }

        const result = db.prepare(
            'INSERT INTO notes (user_id, title, content, category) VALUES (?, ?, ?, ?)'
        ).run(req.user.id, title, content || '', category || 'general');

        const note = db.prepare('SELECT * FROM notes WHERE id = ?').get(result.id);
        res.status(201).json(note);
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});

// PUT /api/notes/:id - Update note
router.put('/:id', (req, res) => {
    try {
        const note = db.prepare('SELECT * FROM notes WHERE id = ? AND user_id = ?').get(req.params.id, req.user.id);

```

```

    if (!note) {
      return res.status(404).json({ error: 'Catatan tidak ditemukan' })
    }

    const { title, content, category, is_archived } = req.body;

    db.prepare(
      'UPDATE notes SET title = ?, content = ?, category = ?, is_archived = ?'
    ).run(
      title || note.title,
      content !== undefined ? content : note.content,
      category || note.category,
      is_archived !== undefined ? (is_archived ? 1 : 0) : note.is_archived,
      req.params.id,
      req.user.id
    );

    const updated = db.prepare('SELECT * FROM notes WHERE id = ?').get(req.params.id);
    res.json(updated);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// DELETE /api/notes/:id - Delete note
router.delete('/:id', (req, res) => {
  try {
    const note = db.prepare('SELECT * FROM notes WHERE id = ? AND user_id = ?').get(req.params.id, req.user.id);

    if (!note) {
      return res.status(404).json({ error: 'Catatan tidak ditemukan' })
    }

    db.prepare('DELETE FROM notes WHERE id = ? AND user_id = ?').run(req.params.id, req.user.id);
    res.json({ message: 'Catatan berhasil dihapus' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

return router;
};

```

public/index.html — Frontend Simplified

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Notes App</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body { font-family: system-ui, sans-serif; background: #f0f2f5; min-height: 100vh; }
    .container { max-width: 900px; margin: 0 auto; padding: 2rem 1rem; }
    .hidden { display: none !important; }

    .card {
      background: white;
      border-radius: 12px;
      padding: 1.5rem;
      box-shadow: 0 2px 8px rgba(0,0,0,0.1);
      margin-bottom: 1rem;
    }

    .btn {
      padding: 0.5rem 1rem;
      border: none;
      border-radius: 8px;
      cursor: pointer;
      font-size: 0.9rem;
      transition: all 0.2s;
    }
    .btn-primary { background: #4f46e5; color: white; }
    .btn-primary:hover { background: #4338ca; }
    .btn-danger { background: #ef4444; color: white; }
    .btn-danger:hover { background: #dc2626; }
    .btn-success { background: #22c55e; color: white; }

    input, textarea, select {
      width: 100%;
      padding: 0.75rem;
      border: 1px solid #ddd;
      border-radius: 8px;
      font-size: 1rem;
      margin-bottom: 1rem;
    }

    .note-grid {
```

```

        display: grid;
        grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
        gap: 1rem;
    }

    .note-card {
        background: white;
        border-radius: 12px;
        padding: 1.25rem;
        box-shadow: 0 2px 8px rgba(0,0,0,0.1);
        border-left: 4px solid #4f46e5;
        transition: transform 0.2s;
    }
    .note-card:hover { transform: translateY(-2px); }
    .note-card .title { font-size: 1.1rem; font-weight: 600; margin-bottom: 0.75rem; }
    .note-card .content { color: #666; font-size: 0.9rem; margin-bottom: 0.75rem; }
    .note-card .meta { font-size: 0.8rem; color: #999; display: flex; gap: 0.5rem; }
    .note-card .actions { display: flex; gap: 0.5rem; margin-top: 0.75rem; }

    .tabs { display: flex; gap: 1rem; margin-bottom: 1.5rem; }
    .tab { padding: 0.5rem 1rem; cursor: pointer; border-bottom: 2px solid transparent; }
    .tab.active { border-bottom-color: #4f46e5; font-weight: 600; }

    .modal-overlay {
        position: fixed; top: 0; left: 0; width: 100%; height: 100%;
        background: rgba(0,0,0,0.5); display: flex; align-items: center;
        justify-content: center; z-index: 1000;
    }
    .modal-content { background: white; border-radius: 12px; padding: 2rem; }

    .auth-container { max-width: 400px; margin: 4rem auto; }
    .header { display: flex; justify-content: space-between; align-items: center; }

    .filter-bar { display: flex; gap: 1rem; margin-bottom: 1.5rem; flex-wrap: wrap; }
    .filter-bar input, .filter-bar select { flex: 1; min-width: 200px; margin: 0; }

    .badge {
        display: inline-block; padding: 0.2rem 0.5rem; border-radius: 4px;
        font-size: 0.75rem; font-weight: 600;
    }
    .badge-general { background: #e0e7ff; color: #4338ca; }
    .badge-work { background: #fef3c7; color: #b45309; }
    .badge-personal { background: #d1fae5; color: #065f46; }
    .badge-archived { background: #e5e7eb; color: #6b7280; }
</style>
</head>

```

```

<body>
  <div class="container">
    <!-- Auth Section -->
    <div id="authSection" class="auth-container">
      <div class="card">
        <div class="tabs">
          <div class="tab active" onclick="showAuthTab('login')">Login
          <div class="tab" onclick="showAuthTab('register')">Register</div>
        </div>

        <div id="loginForm">
          <h2>Login</h2>
          <input type="email" id="loginEmail" placeholder="Email" required>
          <input type="password" id="loginPassword" placeholder="Password">
          <button class="btn btn-primary" onclick="login()">Login</button>
          <div id="loginError" class="hidden" style="color:red;margin-top:10px;>
            <div id="registerForm" class="hidden">
              <h2>Register</h2>
              <input type="text" id="regUsername" placeholder="Username" required>
              <input type="email" id="regEmail" placeholder="Email" required>
              <input type="password" id="regPassword" placeholder="Password">
              <button class="btn btn-success" onclick="register()">Register</button>
              <div id="regError" class="hidden" style="color:red;margin-top:10px;>
            </div>
          </div>
        </div>
      </div>
    </div>

    <!-- Notes Section -->
    <div id="notesSection" class="hidden">
      <div class="header">
        <h1>Notes App</h1>
        <button class="btn btn-danger" onclick="logout()">Logout</button>
      </div>

      <div class="filter-bar">
        <input type="text" id="searchInput" placeholder="Cari catatan...">
        <select id="categoryFilter" onchange="loadNotes()">
          <option value="">Semua Kategori</option>
          <option value="general">General</option>
          <option value="work">Work</option>
          <option value="personal">Personal</option>
        </select>
        <button class="btn btn-primary" onclick="showCreateModal()">+ Tambah</button>
      </div>
    </div>
  </div>

```

```

        <div>
            <button class="btn" onclick="toggleArchived()" id="archiveToggle
        </div>

        <div id="notesContainer" class="note-grid" style="margin-top:1rem;">
    </div>
</div>

<!-- Modal -->
<div id="modalOverlay" class="modal-overlay hidden">
    <div class="modal-content">
        <h2 id="modalTitle">Tambah Catatan</h2>
        <input type="text" id="noteTitle" placeholder="Judul" required>
        <textarea id="noteContent" placeholder="Isi catatan..." rows="5"></t
        <select id="noteCategory">
            <option value="general">General</option>
            <option value="work">Work</option>
            <option value="personal">Personal</option>
        </select>
        <input type="hidden" id="editNoteId">
        <div style="display:flex;gap:0.5rem;">
            <button class="btn btn-primary" onclick="saveNote()">Simpan</but
            <button class="btn" onclick="closeModal()">Batal</button>
        </div>
    </div>
</div>

<script>
    const API = '/api';
    let token = localStorage.getItem('token');
    let showArchived = false;

    // Check auth
    if (token) {
        document.getElementById('authSection').classList.add('hidden');
        document.getElementById('notesSection').classList.remove('hidden');
        loadNotes();
    }

    function showAuthTab(tab) {
        document.querySelectorAll('.tab').forEach(t => t.classList.remove('a
        if (tab === 'login') {
            document.querySelector('.tab:nth-child(1)').classList.add('activ
            document.getElementById('loginForm').classList.remove('hidden');
            document.getElementById('registerForm').classList.add('hidden');

```

```

    } else {
        document.querySelector('.tab:nth-child(2)').classList.add('active');
        document.getElementById('loginForm').classList.add('hidden');
        document.getElementById('registerForm').classList.remove('hidden');
    }
}

async function register() {
    const username = document.getElementById('regUsername').value;
    const email = document.getElementById('regEmail').value;
    const password = document.getElementById('regPassword').value;
    const errorDiv = document.getElementById('regError');

    try {
        const res = await fetch(`${API}/auth/register`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ username, email, password })
        });
        const data = await res.json();

        if (res.ok) {
            alert('Registrasi berhasil! Silakan login.');
            showAuthTab('login');
            document.getElementById('regUsername').value = '';
            document.getElementById('regEmail').value = '';
            document.getElementById('regPassword').value = '';
            errorDiv.classList.add('hidden');
        } else {
            errorDiv.textContent = data.error;
            errorDiv.classList.remove('hidden');
        }
    } catch (err) {
        errorDiv.textContent = 'Network error';
        errorDiv.classList.remove('hidden');
    }
}

async function login() {
    const email = document.getElementById('loginEmail').value;
    const password = document.getElementById('loginPassword').value;
    const errorDiv = document.getElementById('loginError');

    try {
        const res = await fetch(`${API}/auth/login`, {
            method: 'POST',

```

```

        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ email, password })
    });
    const data = await res.json();

    if (res.ok) {
        token = data.token;
        localStorage.setItem('token', token);
        localStorage.setItem('user', JSON.stringify(data.user));
        document.getElementById('authSection').classList.add('hidden');
        document.getElementById('notesSection').classList.remove('hidden');
        loadNotes();
        errorDiv.classList.add('hidden');
    } else {
        errorDiv.textContent = data.error;
        errorDiv.classList.remove('hidden');
    }
} catch (err) {
    errorDiv.textContent = 'Network error';
    errorDiv.classList.remove('hidden');
}
}

function logout() {
    localStorage.removeItem('token');
    localStorage.removeItem('user');
    token = null;
    document.getElementById('authSection').classList.remove('hidden');
    document.getElementById('notesSection').classList.add('hidden');
}

async function loadNotes() {
    const search = document.getElementById('searchInput').value;
    const category = document.getElementById('categoryFilter').value;
    const container = document.getElementById('notesContainer');

    let url = `${API}/notes?archived=${showArchived}`;
    if (search) url += `&search=${encodeURIComponent(search)}`;
    if (category) url += `&category=${encodeURIComponent(category)}`;

    try {
        const res = await fetch(url, {
            headers: { 'Authorization': `Bearer ${token}` }
        });
        const notes = await res.json();
    }

```

```

    if (notes.length === 0) {
      container.innerHTML = '<p style="text-align:center;color:#99'
      return;
    }

    container.innerHTML = notes.map(note => `
      <div class="note-card">
        <div class="title">${escapeHtml(note.title)}</div>
        <div class="content">${escapeHtml(note.content?.substring(
        <div class="meta">
          <span class="badge badge-${note.category}">${note.ca
          <span>${new Date(note.updated_at).toLocaleDateString
        </div>
        <div class="actions">
          <button class="btn btn-primary" onclick="editNote(${
          <button class="btn btn-danger" onclick="deleteNote($
          <button class="btn" onclick="toggleArchive(${note.id
            ${note.is_archived ? 'Aktifkan' : 'Arsipkan'}
          </button>
        </div>
      </div>
    `).join('');
  } catch (err) {
    container.innerHTML = '<p style="color:red;">Gagal memuat catata
  }
}

function toggleArchived() {
  showArchived = !showArchived;
  document.getElementById('archiveToggle').textContent = showArchived
  loadNotes();
}

function showCreateModal() {
  document.getElementById('modalTitle').textContent = 'Tambah Catatan'
  document.getElementById('noteTitle').value = '';
  document.getElementById('noteContent').value = '';
  document.getElementById('noteCategory').value = 'general';
  document.getElementById('editNoteId').value = '';
  document.getElementById('modalOverlay').classList.remove('hidden');
}

async function editNote(id) {
  try {
    const res = await fetch(`${API}/notes/${id}`, {
      headers: { 'Authorization': `Bearer ${token}` }
    }

```

```

    });
    const note = await res.json();

    document.getElementById('modalTitle').textContent = 'Edit Catatan';
    document.getElementById('noteTitle').value = note.title;
    document.getElementById('noteContent').value = note.content || '';
    document.getElementById('noteCategory').value = note.category;
    document.getElementById('editNoteId').value = note.id;
    document.getElementById('modalOverlay').classList.remove('hidden');
  } catch (err) {
    alert('Gagal memuat catatan');
  }
}

async function saveNote() {
  const title = document.getElementById('noteTitle').value;
  const content = document.getElementById('noteContent').value;
  const category = document.getElementById('noteCategory').value;
  const editId = document.getElementById('editNoteId').value;

  if (!title) return alert('Judul wajib diisi');

  const url = editId ? `${API}/notes/${editId}` : `${API}/notes`;
  const method = editId ? 'PUT' : 'POST';

  try {
    const res = await fetch(url, {
      method,
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${token}`
      },
      body: JSON.stringify({ title, content, category })
    });

    if (res.ok) {
      closeModal();
      loadNotes();
    } else {
      const data = await res.json();
      alert(data.error);
    }
  } catch (err) {
    alert('Gagal menyimpan catatan');
  }
}

```

```

async function deleteNote(id) {
  if (!confirm('Hapus catatan ini?')) return;

  try {
    const res = await fetch(`${API}/notes/${id}`, {
      method: 'DELETE',
      headers: { 'Authorization': `Bearer ${token}` }
    });

    if (res.ok) {
      loadNotes();
    } else {
      const data = await res.json();
      alert(data.error);
    }
  } catch (err) {
    alert('Gagal menghapus catatan');
  }
}

async function toggleArchive(id, currentState) {
  try {
    const res = await fetch(`${API}/notes/${id}`, {
      method: 'PUT',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${token}`
      },
      body: JSON.stringify({ is_archived: !currentState })
    });

    if (res.ok) {
      loadNotes();
    }
  } catch (err) {
    alert('Gagal mengubah status arsip');
  }
}

function closeModal() {
  document.getElementById('modalOverlay').classList.add('hidden');
}

function escapeHtml(str) {
  const div = document.createElement('div');

```

```

        div.textContent = str;
        return div.innerHTML;
    }

    // Close modal on overlay click
    document.getElementById('modalOverlay').addEventListener('click', function(e) {
        if (e.target === this) closeModal();
    });
</script>
</body>
</html>

```

□ Latihan

Latihan 1: Kerjakan Soal 1 (Web Profil Perusahaan)

Timer: 4 jam

Kerjakan sesuai spesifikasi Soal 01. Gunakan timer. Setelah selesai: 1. Self-assessment menggunakan scoring checklist 2. Catat fitur mana saja yang belum selesai 3. Analisis: kenapa tidak selesai? (time management? skill gap? bug?)

Latihan 2: Kerjakan Soal 2 (Sistem Perpustakaan)

Timer: 4 jam

Fokus pada: - Database schema yang benar (3 tabel, relasi, foreign key) - CRUD dengan prepared statements - Search dengan AJAX - Peminjaman dengan validasi stok

Latihan 3: Kerjakan Soal 3 (Notes API)

Timer: 4 jam

Fokus pada: - REST API best practices (HTTP methods, status codes) - JWT authentication - Ownership validation - Frontend integration dengan fetch API

Latihan 4: Full Mock Test (Simulasi LKS)

Timer: 6 jam

Pilih salah satu soal (1/2/3). Kerjakan dalam kondisi simulasi: 1. No internet browsing 2. No AI tools 3. Strict timer 4. Submit hasil dalam folder ZIP 5. Evaluasi skor menggunakan scoring checklist

☐ **Checklist Sesi 03**

- ☐ Mengerjakan minimal 1 soal mock LKS (4 jam)
- ☐ Menggunakan timer sesuai durasi
- ☐ Self-assessment dengan scoring checklist
- ☐ Identifikasi area yang perlu diperbaiki
- ☐ Mencoba full mock test (6 jam) jika ada waktu
- ☐ Produk akhir: folder proyek lengkap dengan dokumentasi

☐ **Selamat! Kamu sudah siap untuk LKS Web Technology!**